```
  return item in
node.shop.items
#2

nn = fNN(root, customerLocation, hasItem(X))
                                  #3
```

#1 A generic version for `hasItem` takes the item as well as the node as arguments.
#2 Check if the list of items associated with a shop contains `item`.
#3 We pass a curried instance of `hasItem` to the `fNN` method, to find the closest point whose shop has item X in the inventory (here `X` is supposed to be a variable containing the item we look for).

Unfortunately, while this filtering mechanism allows to filter out retailers that doesn't have the items we need, it isn't powerful enough to let us decide that it is best to choose a closer retailer with higher shipment costs over a cheaper one that is twice as far away, and in general it can't deal with complex conditions comparing different solutions.

11.2.2 Complex Decisions

If what we really need is not just filtering shops according to some criteria, but also choosing the best options among them, then we have to seek for a more powerful mechanism.

We mainly have two choices:

· Either use n-NearestNeighbor, retrieve a list of `n` shops that satisfy our criteria, and then process this list to